

M1.1: 单例模式: DCL 与 volatile 防御指令重排

Theory: 保证 JVM 运行期间某类仅有一个实例。双重检查锁定 (Double-Checked Locking, DCL) 在减少同步开销的前提下实现了延迟加载。

Details: 声明实例变量时必须加 'volatile', 因为 'instance = new Singleton()' 并非原子操作, 而是分三步: 1. 分配堆内存; 2. 调用构造函数初始化; 3. 将引用指向内存地址。无 'volatile' 时, JVM 可能会指令重排执行 1->3->2, 导致多线程在高频调用下获取到未完全初始化的“半初始化对象”。

Trade-off: DCL 需要双重 null 校验加 'synchronized' 同步块。若觉得代码复杂, 可采用静态内部类形式 (Holder 模式), 利用 JVM 类加载器的初始化时锁机制实现天然的延迟加载与线程安全。

M1.4: 外观与享元模式减少内存开销

Theory: 外观模式 (Facade) 为复杂的子系统提供一个统一的简化高层接口; 享元模式 (Flyweight) 通过共享池机制, 复用大量细粒度的相同对象, 以减少堆内存消耗。

Details: 外观模式不提供新的功能, 仅负责子系统的方法编排。享元模式将对象状态分为内部状态 (共享且不随环境改变) 与外部状态 (依赖具体环境), 使用 Map 缓存已创建的内部状态对象, 如 Java 的 'Integer.valueOf' 缓存 (-128 到 127)。

Trade-off: 享元模式要求严格区分内外状态, 且在读写时可能因同步问题带来额外的 CPU 查找开销。如果被复用对象的生命周期极短, 缓存池的维护可能会反而阻碍 JVM 垃圾回收。

M2.2: 模板方法与命令模式规范事务流水线

Theory: 模板方法在超类中定义算法骨架, 允许子类在不改变结构的前提下重写某些特定步骤; 命令模式将请求封装为独立的命令对象, 实现请求发送者与具体执行者解耦, 方便支持撤销和重做。

Details: 模板方法利用继承实现, 父类方法声明为 'final' 控制执行流程, 变动步骤声明为 'protected abstract' 钩子方法。命令模式将接收者 (Receiver) 封装在命令 (Command) 中, 通过调用者 (Invoker) 控制命令的存储、执行与历史撤销栈。

Trade-off: 模板方法采用继承机制, 如果算法骨架本身需要频繁变更, 会导致大量的子类被迫修改, 违背灵活组合的原则。命令模式则可能因海量的细粒度操作产生大量的命令对象, 增加 JVM 垃圾回收压力。

M2.4: 中介者与备忘录模式封装内部交互与回滚

Theory: 中介者模式通过一个中介对象封装一系列对象间的复杂交互, 使对象间不再显式引用, 实现多对多到一对多降维; 备忘录模式在不破坏封装性的前提下, 捕获一个对象的内部状态, 以便在未来恢复到该历史状态。

Details: 中介者持有所有同事类 (Colleague) 的引用, 同事类仅与中介者通信, 由中介者协调各同事的交互逻辑。备忘录模式包含三要素: 原发器 (Originator, 创建与恢复备忘录)、备忘录 (Memento, 保存状态只读)、负责人 (Caretaker, 管理备忘录历史栈)。

Trade-off: 备忘录模式在高频大对象状态备份下, 会占用巨大的内存资源。需要设计定时的清理或版本淘汰策略; 中介者模式则容易让中介者类本身发生“上帝类”膨胀, 聚集了子系统所有的复杂性。

M1.2: 工厂方法与抽象工厂模式

Theory: 经典 GoF 创建型模式。工厂方法定义了创建单一对象的接口, 由子类决定具体实例化哪个类; 抽象工厂模式则提供创建一系列相互依赖的“产品族”的接口, 而无需指定具体类。

Details: 工厂方法将对对象创建的逻辑延迟到具体子类中, 遵循开闭原则。抽象工厂则包含多个工厂方法, 可构建包含数据库连接、语句解析器及事务管理器在内的完整产品生态族。

Trade-off: 引入工厂模式会产生大量的具体类文件, 增加系统层级和理解开销。如果创建的对象非常单一且没有未来扩展需求, 使用直接 'new' 依然是最清爽的选择。

M1.5: 观察者模式与发布-订阅机制事件解耦

Theory: 定义对象间的一对多依赖关系, 当一个对象的状态发生改变时, 所有依赖于它的观察者对象都会得到通知并自动被调用更新。

Details: 被观察者 (Subject) 维护一个观察者 (Observer) 列表, 提供注册、注销和 'notifyObservers()' 接口。更新通常可选择“拉模式”(仅通知有变, 由观察者反向拉数据) 或“推模式”(通知时携带完整的状态变化数据包)。

Trade-off: 默认的观察者模式多为单线程同步调用。如果某个观察者在处理更新时发生大延迟或死锁, 会导致被观察者的整个主流程被卡死。在大型分布式系统中, 应将其拆解为基于消息队列 (MQ) 的异步发布订阅。

M1.3: 适配器与装饰器模式结构装配

Theory: 适配器模式用于解决两个已有接口不兼容的问题。充当转换介质; 装饰器模式旨在不改变原有对象接口的前提下, 通过组合动态地为对象包装、添加新的职责。

Details: 适配器主要是做数据与调用接口的格式转换; 装饰器持有被装饰者的引用 (通过构造函数注入), 其接口方法内部通常先调用被装饰者的方法, 再执行自身的增强逻辑, 如 Java I/O 流中的 'BufferedInputStream(FileInputStream)'。

Trade-off: 装饰器模式支持无限层的嵌套包装以获得复合行为, 但如果包装的层级过深, 会增加代码调试和异常堆栈追踪的难度, 引发内存堆栈臃肿。

M2.1: 策略与状态模式消除多分支判定

Theory: 策略模式定义一系列可互换的算法, 并将具体实现算法封装在独立的类中, 使其独立于调用客户端; 状态模式允许对象在内部状态改变时改变其行为, 看起来像修改了其类。

Details: 策略模式通常在客户端通过构造注入或上下文环境选择性调用。状态模式则将状态转移逻辑分散到各个具体状态类中。状态类持有上下文对象引用, 并在对应行为执行完后自动切换上下文的当前状态。

Trade-off: 策略和状态类的大量拆分会导致系统中产生过多的类文件。当状态转移规则简单 (仅 2-3 个状态) 时, 使用常规的条件判断 (if-else) 往往比引入状态模式更加精简。

M2.3: 责任链模式: 网关请求拦截过滤器管道

Theory: 避免请求发送者和接收者耦合, 将多个处理节点连接成链, 请求沿着链传递, 直到有节点处理它或链条结束。

Details: 每个处理器 (Handler) 都持有下一个处理器的引用。典型设计有“单向断链 responsibility”(一个处理完即中止) 和“双向多重拦截 filter”(每个节点在请求前和响应后双向处理, 如 Servlet 的 'FilterChain' / Spring MVC 的 'HandlerInterceptor')。

Trade-off: 链条过长会显著增加请求在堆栈中的流转开销。如果在链中存在递归调用且链条未被正确截断, 可能引发栈溢出崩溃 ('StackOverflowError')。

M2.5: 数据传输对象 (DTO) 与业务实体解耦设计

Theory: 用于跨越物理边界或网络协议层传输数据。DTO 仅包含无行为的纯字段 (Getter/Setter), 用于隔离底层数据库实体 (Entity) 的内部变化, 防止库表敏感字段泄露。

Details: DTO 是与网络层协议高度绑定的扁平化结构。业务实体 (Entity) 则是包含行为和业务逻辑的领域模型。两者之间的装配可采用 MapStruct 这种在编译期直接生成 'setter/getter' 映射代码的工具, 避免反射导致的 CPU 开销。

Trade-off: 引入 DTO 和 Entity 的频繁转换, 会导致系统中出现大量重复的转换代码与繁多的 DTO 对象。在小型单表增删改查 (CRUD) 项目中, 直接将实体类穿透暴露往往更加简单。

M3.1: 数据访问对象（DAO）与仓储（Repository）模式

Theory: 数据访问抽象模式。DAO 封装了对底层特定物理库表的 CRUD 操作；仓储模式（Repository）是领域驱动设计（DDD）的战术模式，为上层业务领域提供统一的、面向集合的持久化抽象接口。

Details: DAO 粒度非常细，通常一个表对应一个 DAO。Repository 粒度较粗，以聚合根为单位进行持久化，其底层实现可能组合调用了多个 DAO，并将底层零散数据装配为完整的聚合根领域模型。

Trade-off: Repository 提供了极佳的领域一致性与存储屏蔽。但在高频大批量写入或大数据报表查询场景下，Repository 复杂的对象拼装和多次级联数据库查询操作会产生严重的性能瓶颈，此时必须绕过 Repository 直接执行批量 SQL。

M3.2: 拦截过滤器（Intercepting Filter）与前端控制器

Theory: 企业级 Web 框架的核心架构。前端控制器（Front Controller）提供集中式的请求路由处理分发入口；拦截过滤器（Filter）在请求到达前端控制器前后，对请求和响应做统一的横切关注点处理（如权限、编码、限流）。

Details: 典型实现如 Spring MVC 的 ‘DispatcherServlet’ 作为前端控制器，通过 ‘HandlerMapping’ 路由分发；配合 ‘HandlerExecutionChain’ 中的拦截器管道，在目标 Controller 执行前、后以及页面渲染完毕后执行横切逻辑。

Trade-off: 前端控制器作为系统的唯一流量入口，面临极高的并发压力。如果拦截过滤器内部包含阻塞式的外部 RPC 调用（如同步向用户中心校验 Token），会导致前端控制器的线程池瞬间占满。

M3.3: 微服务断路器与故障转移降级设计

Theory: 分布式系统防雪崩的核心模式。当下游接口依赖出现高延迟或异常报错时，主动开启熔断保护，快速返回失败或备选 Fallback 数据，防止故障在服务链路中级联蔓延。

Details: 熔断器状态机包含：1. Closed（正常）；2. Open（异常率达到阈值，触发限流并走 Fallback 快速失败）；3. Half-Open（冷却后，放行极少流量试探，成功则归于 Closed，失败则重新退回 Open）。

Trade-off: 故障转移（Failover）虽然保全了系统的整体可用性，但会造成业务有损。在编写降级 Fallback 逻辑时，应确保返回的数据是高安全性的缓存数据或无损兜底，防止将错误脏数据写入前端。

M3.4: CQRS 与事件溯源（Event Sourcing）高扩展模型

Theory: 完全分离系统的读写职责。命令端（Command）处理写操作，追求极高的写性能与数据一致性；查询端（Query）处理读操作，针对特定视图视图进行极佳的索引优化。

Details: 事件溯源不保存实体的当前状态，而是将实体生命周期中的每一个状态变更封装为“事件”，顺序追加写入 Event Store。当需要读取当前状态时，通过重放（Replay）历史事件重构出最新状态，并异步投影到只读查询库。

Trade-off: 事件溯源提供了天然的审计日志和追溯性，但其开发复杂度极高。事件数量极其巨大时，Replay 耗时会呈线性增长，必须引入“快照（Snapshot）”机制，每隔一万个事件生成一个局部快照以截断重放流程。

M3.5: Java 线程模型与 OS 内核线程 1:1 映射

Theory: 在 HotSpot 虚拟机上，Java 中的线程（‘java.lang.Thread’）与操作系统的内核线程（Kernel Thread）是一对一映射的。

Details: Java 线程的创建、销毁、暂停和调度完全委托给底层 OS 调度器。每个 Java 线程拥有私有的 JVM 线程栈，默认分配 1MB 空间，用于存储局部变量、操作数栈和方法调用栈帧。

Trade-off: 由于是 1:1 映射，线程的创建和销毁涉及高昂的物理系统调用，并且在抢占 CPU 时会产生剧烈的内核上下文切换开销。因此，Java 严禁高频创建/销毁线程，必须使用线程池管理。

M4.2: ThreadPoolExecutor 线程池工作线程复用与饱和策略

Theory: 线程池通过池化机制，复用一定数量的工作线程，以降低线程频繁创建和销毁产生的开销。

Details: 核心参数包括：核心线程数（‘corePoolSize’）、最大线程数（‘maximumPoolSize’）、等待队列（‘workQueue’）。提交任务时，如果当前线程数少于核心线程，新建工作线程；如果已满，将任务排入工作队列；如果工作队列已满且当前线程数少于最大线程数，新建临时工作线程；如果全部爆满，则触发饱和和拒绝策略（如 ‘AbortPolicy’，‘CallerRunsPolicy’）。

Trade-off: 等待队列严禁使用无界队列（如默认的 ‘LinkedBlockingQueue’）。在高负荷下，无界队列会无限积压任务，导致 JVM 堆内存耗尽，触发 ‘OutOfMemoryError’ 彻底崩溃。

M4.4: ReentrantLock 与 AQS CLH 双向队列同步状态机

Theory: 显式锁的基石。基于 AQS（AbstractQueuedSynchronizer）实现，利用一个 volatile 状态变量 ‘state’ 表示锁状态，以及一个 CLH 双向 FIFO 等待队列管理抢锁失败的线程。

Details: 抢锁时，线程通过 CAS 尝试将 ‘state’ 从 0 修改为 1。若失败，线程会被封装为 Node 节点，通过 CAS 自旋追加插入到 AQS 双向链表队列的末尾，并调用 ‘LockSupport.park(this)’ 挂起，等待前驱节点释放锁后将其唤醒（unpark）。

Trade-off: 显式锁支持公平与非公平模式。非公平锁允许刚来的线程插队抢锁，吞吐量较高，但可能导致队列中的线程发生“饥饿”（长时间得不到执行）；公平锁则杜绝了饥饿，但频繁的挂起与唤醒会导致整体上下文切换开销飙升。

M4.1: JMM 核心机制与 volatile 内存屏障

Theory: Java 内存模型（JMM）规范了 JVM 如何控制工作线程与主内存之间的交互，以保证可见性、有序性和原子性。

Details: ‘volatile’ 保证可见性与有序性。JMM 规范在编译 ‘volatile’ 读写时会插入硬件级的内存屏障（Memory Barrier，如 ‘StoreStore’，‘StoreLoad’，‘LoadLoad’，‘LoadStore’），物理强制冲刷当前 CPU 的 Store Buffer，并使其他 CPU 缓存行失效。

Trade-off: ‘volatile’ 仅保证变量本身的可见性和有序性，但不保证复合操作（如 ‘count++’，包含了读取、加一、写入三步）的原子性。若要保证复合操作的线程安全，必须使用 ‘Atomic’ 原子类（CAS）或同步锁。

M4.3: ThreadLocal 线程局部存储与内存泄漏防御

Theory: 为每个线程提供独立的变量副本，实现线程级别的变量隔离，从而避免多线程间的资源抢占和同步开销。

Details: 线程类 ‘Thread’ 内部持有一个 ‘ThreadLocalMap’，其 entry 的 Key 是指向 ‘ThreadLocal’ 对象的弱引用（‘WeakReference’），Value 是存储的实际对象引用。

Trade-off: 弱引用的 Key 在下一次垃圾回收时会被自动回收，使 Key 变为 null。但如果线程长期运行（如线程池中的线程被复用），该 entry 的 Value 由于是强引用，会一直沿着线程对象的强引用链向下存活，造成内存泄漏。必须在每次使用完 ThreadLocal 后，显式调用 ‘remove()’ 清理。

M5.1: CAS 无锁原子操作与 ABA 冲突防范

Theory: 乐观锁的核心技术。Compare-And-Swap 通过硬件 CPU 提供的原子指令（如 x86 的 ‘cmpxchg’），在不使用互斥锁的前提下实现对共享变量的安全更新。

Details: CAS 包含三个操作数：内存位置 V、预期原值 A 和新值 B。只有当内存 V 中的当前值等于预期值 A 时，才将该位置更新为新值 B，否则不断自旋重试。

Trade-off: CAS 无法识别 ABA 问题：即变量在读取时是 A，在准备写入时也是 A，但在此期间被其他线程先改为了 B 又改回了 A。虽然最终值一样，但对于涉及对象引用的场景可能产生逻辑漏洞。解决方案是使用 ‘AtomicStampedReference’ 引入单调递增的版本号（Stamp）。

M5.4: 响应式编程流规约与有界背压机制

Theory: 响应式编程（Reactive Programming）致力于解决异步非阻塞数据流的传递问题。背压（Backpressure）机制在数据发布者（Publisher）与订阅者（Subscriber）之间建立了一种双向有界的反馈通道。

Details: 在 Reactive Streams 规范中，订阅者通过调用 ‘Subscription.request(n)’ 显式通知发布者自身当前最大可以处理的数据元素个数 n。发布者被严格限制在得到通知后方可推送不多于 n 个数据，从而保护内存资源受限的订阅者不被冲垮。

Trade-off: 背压需要完整链路的库（从网关、应用层到数据库驱动）全部遵循响应式规范。一旦链路中某个非响应式三方库执行了阻塞的同步调用（如 JDBC 查询），背压通道会彻底失效，形成上游大内存积压。

M5.2: ConcurrentHashMap 并发容器与红黑树平衡

Theory: 在高并发下提供极高吞吐量的线程安全 Map 容器。

Details: 相比于早期全表锁的 ‘HashTable’ 或 1.7 的 Segment 分段锁，1.8 的 ‘ConcurrentHashMap’ 采用 “行级锁优化”：直接锁桶节点。内部使用 Node 数组加链表/红黑树。当发生哈希冲突进行写操作时，仅使用 ‘synchronized’ 锁定当前桶的头节点（Head Node），不影响其他桶的并发读写；当链表长度超过 8 且数组长度超过 64 时，链表自动转换为红黑树。

Trade-off: ConcurrentHashMap 在扩容（Transfer）时需要进行复杂的桶迁移，此时会让多个写线程共同协作进行数据搬迁，这会在扩容期间造成短暂的响应延迟抖动。

M5.3: 死锁（Deadlock）环路判定与预防规避

Theory: 多线程由于争夺共享资源而造成的陷入无限互相等待的死锁状态。死锁发生的四个必要条件：互斥、占有且等待、非抢占和循环等待。

Details: JVM 内置死锁检测主要通过追踪等待关系图（Wait-for Graph）是否形成环路。如果 Thread 1 占有 Lock A 并在等待 Lock B，而 Thread 2 占有 Lock B 并在等待 Lock A，图谱形成闭环，触发死锁监控报告。

Trade-off: 规避死锁的黄金法则包括：1. 锁顺序化（必须以相同的顺序获取锁，防止形成逆向等待环）；2. 超时退出（使用 ‘ReentrantLock.tryLock(timeout)’ 替代死等，防范无限挂起）；3. 减小锁粒度（使用读写锁或并发容器替代大排他锁）。

M5.4: 响应式编程流规约与有界背压机制

Theory: 响应式编程（Reactive Programming）致力于解决异步非阻塞数据流的传递问题。背压（Backpressure）机制在数据发布者（Publisher）与订阅者（Subscriber）之间建立了一种双向有界的反馈通道。

Details: 在 Reactive Streams 规范中，订阅者通过调用 ‘Subscription.request(n)’ 显式通知发布者自身当前最大可以处理的数据元素个数 n。发布者被严格限制在得到通知后方可推送不多于 n 个数据，从而保护内存资源受限的订阅者不被冲垮。

Trade-off: 背压需要完整链路的库（从网关、应用层到数据库驱动）全部遵循响应式规范。一旦链路中某个非响应式三方库执行了阻塞的同步调用（如 JDBC 查询），背压通道会彻底失效，形成上游大内存积压。

M6.1: CompletableFuture 多任务异步流水线编排

Theory: Java 8 引入的异步任务编排利器。支持对多个异步计算任务进行串联（thenApply）、并联（thenCombine）和任意或全部完成条件下的流水线组合执行。

Details: 底层依靠 ForkJoinPool 线程池执行。支持通过 ‘thenCompose’ 实现扁平化任务链（前任务返回 CompletableFuture 作为后任务输入），以及 ‘exceptionally’ 捕获拦截异步计算过程中的异常。

Trade-off: CompletableFuture 默认共享全局的 ‘ForkJoinPool.commonPool()’ 线程池。一旦某一步骤执行了耗时很长的阻塞 I/O，会把全局共享的公用池资源全部耗尽，导致系统所有异步任务都在排队挂起。在生产环境下，必须为不同的业务场景配置专属的线程池。

M6.2: ForkJoinPool 并行框架与工作窃取算法

Theory: 大规模并行计算的调度引擎。适合将大型任务递归地分割（Fork）为小任务，然后合并（Join）结果的并行场景。

Details: 工作窃取（Work-Stealing）算法中，每个工作线程（ForkJoinWorkerThread）都维护一个私有的双端队列（Deque）来存储分配给自己的任务。工作线程总是以 LIFO 顺序从自己队列的“头部”获取任务执行；当自己的队列清空后，工作线程会扮演“小偷”角色，以 FIFO 顺序从其他忙碌线程队列的“尾部”窃取一个任务来帮忙执行，极大地消除了负载不平衡。

Trade-off: 双端队列的头尾分流设计虽然避免了绝大多数锁竞争，但任务的不断递归 Fork 会产生大量的轻量级子任务对象，如果堆内存本身比较紧张，会引发频繁的 JVM 垃圾回收，抵消并发带来的速度提升。

M6.3: Card 27 Fallback Title

Placeholder text for compiler robustness.

M6.4: Card 28 Fallback Title

Placeholder text for compiler robustness.

Zone T1: Java Concurrency APIs

Proxy.newProxyInstance(), AbstractQueuedSynchronizer.compareAndSetState(), ThreadPoolExecutor.execute(), CompletableFuture.allOf(), ThreadLocal.remove(), ConcurrentHashMap.putVal(), ForkJoinPool.commonPool()

Zone T2: System Concurrency Faults

out_of_memory_metaspace, concurrency_deadlock, thread_pool_exhaustion, thread_local_leak, aba_problem, concurrent_modification, race_condition